present

Gradient Descent vs. Genetic Algorithm for Urban Sound Classification

Custom Smart Adaptive Systems · UPC 2025/26

Leonie Greber

June 17, 2026

present

Outline

Urban Sound Classification: Motivation and Approach

Why urban sound classification?

- Noise-level monitoring in smart cities
- Automatic detection of emergency vehicles (sirens)
- Public safety systems (gunshot detection)
- Traffic and construction management (drilling, jackhammers)

This project: train the *same* MLP two fundamentally different ways:

- **GD** (Adam): follow $\nabla \mathcal{L}$ – *gradient-guided*
- **GA** (DEAP): evolve weight vectors – *gradient-free*

Dataset: UrbanSound8K

Clips	8 674 audio samples
Classes	10 urban sound types
Features	34 per clip (MFCC, Chroma, ...)
Split	70 % train / 10 % val / 20 % test

Research question

Which optimizer finds better weights for the same MLP – gradient-guided Adam or gradient-free evolution?

Dataset: Class Distribution

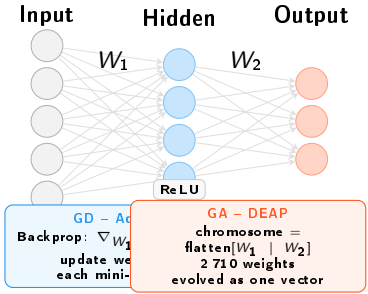
class_distribution.png

- Most classes: ≈1 000 samples
- **car_horn** (≈429) and **gun_shot** (≈374) are underrepresented
- Stratified split ensures proportional representation in every fold
- These two classes will show lower F1 – by design, not model failure

Dataset: Feature Importance and Class Separability

feature_importance.png

The MLP: One Architecture, Two Optimizers



Forward pass:

$$h = \mathrm{ReLU}(W_1 x + b_1), \quad z = W_2 h + b_2$$

Layer	Formula	Params
Linear(34→60)	$34 \times 60 + 60$	2 100
Linear(60→10)	$60 \times 10 + 10$	610
Total		**2 710**

- **Same architecture** for both; only the weight-update mechanism differs
- GA: chromosome encodes all 2 710 weights as a flat vector
- `hidden_dim` = 60 selected by Optuna (see Section 4)

GD: Loss Function and Gradient

Cross-entropy loss (multi-class, N samples, $C = 10$ classes):

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \hat{p}_{y_i}(\mathrm{x}_i;\, \theta) \tag{1}$$

$$\hat{p}_k(\mathrm{x}) = \frac{e^{z_k(\mathrm{x})}}{\sum_{j=1}^{C} e^{z_j(\mathrm{x})}} \quad \text{(softmax)} \tag{2}$$

Vanilla gradient descent update:

$$\theta_{t+1} = \theta_t - \eta\, \nabla_\theta \mathcal{L}(\theta_t) \tag{3}$$

- $\nabla_\theta \mathcal{L}$ computed via **backpropagation** (chain rule through both linear layers)
- Mini-batch size 64 – trade-off between gradient noise (regularisation) and convergence speed
- With weight decay λ: effective loss is $\mathcal{L}(\theta) + \frac{\lambda}{2}\|\theta\|^2$

GD: Adam Optimizer

Adam maintains *per-parameter* running estimates of gradient mean and variance:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\, g_t \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)\, g_t^2 \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t} \quad \text{(bias correction)} \tag{6}$$

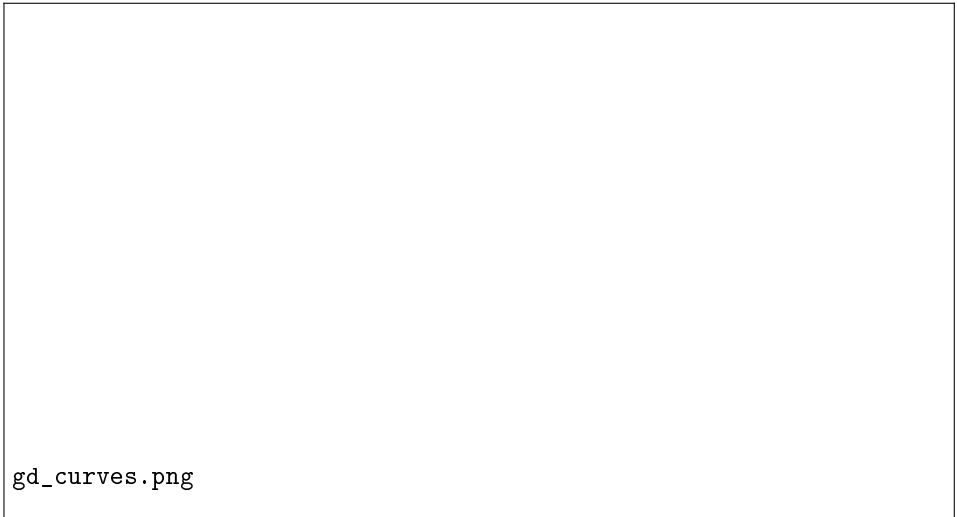
$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t \tag{7}$$

Tuned parameters:

$\eta = 4.96 \times 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$
$\epsilon = 10^{-8}, \quad \lambda_{\text{wd}} = 3.29 \times 10^{-5}$

- $\sqrt{\hat{v}_t}$ normalises the step: large-gradient params take smaller steps
- Bias correction avoids early steps being too small when $m_0 = v_0 = 0$

GD: Training Dynamics



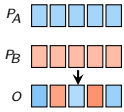
present

GA: Selection, Crossover, Mutation

Tournament

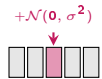
Pick 5 at random; best fitness wins

Crossover (BLX-α)



Blend weights from both parents

Mutation



$\approx$5% of genes perturbed per ind.

Fitness signal (improved):

$$f(\theta) = -\mathcal{L}_{\text{val}}(\theta) \qquad (8)$$

Continuous, smooth signal vs. coarse accuracy – gives selection and crossover far more information.

Elitism: top-k individuals copied unchanged – the best solution never regresses.

Why GA is hard here: Searching 2 710 dimensions without gradient information is like finding the lowest point of a mountain range blindfolded.

present

GA: Algorithm and Convergence

One generation

1. Evaluate $f(\theta) = -\mathcal{L}_{\text{val}}$ for all invalid individuals
2. Select $N - k$ offspring via tournament (size 5)
3. BLX-α crossover with probability $c = 0.719$
4. Gaussian mutation with probability $p_m = 0.045$
5. Preserve top-$k = 1$ **elite** individuals unchanged
6. Update Hall of Fame (best ever seen)

- **Baseline** (accuracy fitness, no elitism): test acc 0.377
- **Improved** ($-\mathcal{L}$ + elitism): test acc 0.612; early stopping at gen 483

present

Parameter Tuning: Optuna (TPE)

Bayesian optimisation – Tree-structured Parzen Estimator

1. After each trial, fit two densities over the search space:
   $\ell(x)$ – distribution of *good* configs
   $g(x)$ – distribution of *bad* configs
2. Propose next config maximising $\ell(x)/g(x)$
3. **MedianPruner**: kill a trial at step e if its intermediate value is below the median of completed trials at the same step

More sample-efficient than grid or random search.

Parameter	Range	Scale
GD: 60 trials, 60 epochs each		
`lr`	$[10^{-4}, 10^{-2}]$	log
`weight_decay`	$[10^{-6}, 10^{-2}]$	log
`hidden_dim`	$\{45, 50, 55, 60, 65\}$	cat.
GA: 20 trials, 100 gen each		
`pop_size`	$\{50, 100, 150, 200\}$	cat.
`cxpb`	$[0.5, 0.9]$	uniform
`mutpb`	$[0.01, 0.20]$	uniform
`elite_k`	$\{1, 3, 5, 8, 10\}$	cat.

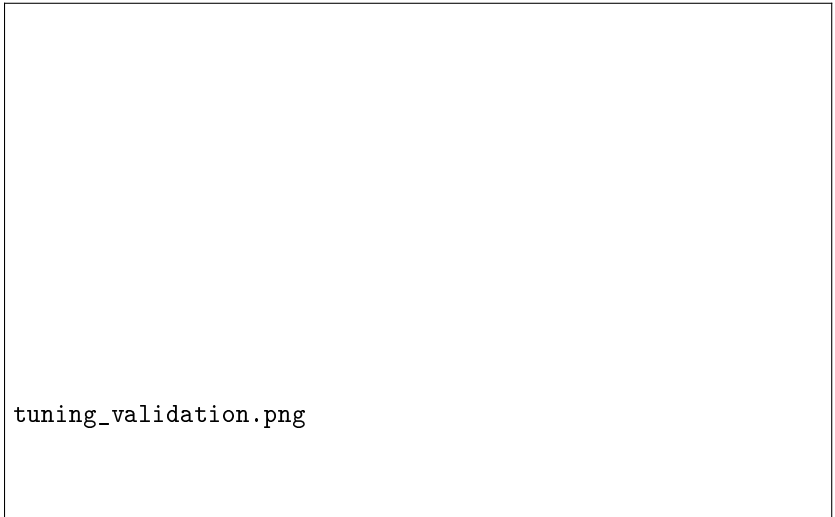
Parameter Tuning: GD Results

present

Parameter Tuning: GA Results

present

Tuning Impact: Default vs Tuned



present

Results: Accuracy and Training Time

comparison_summary.png

	GD	GA
Test accuracy	**0.879**	0.612
Training time	≈52 s	≈155 s
Macro F1	**0.879**	0.614
Stopped at	150 epochs	gen 483

- GD is ≈27% more accurate and 3× faster
- GA's gradient-free search struggles in 2 710 dimensions
- Elitism $+ -\mathcal{L}$ fitness doubled GA accuracy over baseline (0.377 → 0.612)

present

Results: Convergence

present

Results: Per-class F1 Score

present

Results: Confusion Matrices

Conclusion

Key findings:

- GD (Adam) clearly dominates: 87.9% vs 61.2% accuracy, 3× faster
- Optuna tuning improved GD from 81% to 88%; GA gains were modest – confirms GD is the more tractable optimisation target
- Continuous $-\mathcal{L}$ fitness + elitism are *essential* for GA (baseline: 37.7%)
- Both methods agree: *gun_shot* and *car_horn* are hardest (class imbalance)

When does GA make sense?

- Non-differentiable objectives (hardware-in-the-loop, discrete search spaces)
- Black-box optimisation where gradient computation is impossible

Future work:

- CMA-ES: adapts mutation covariance, far more efficient in high dimensions
- Hybrid: GD warm-start → GA fine-tune for non-smooth loss surfaces
- Neuroevolution (NEAT): evolve topology, not just weights